Week 3 Milestone Dataset Charter V1

April 2026

1 Executive Summary

This report pulls together the seven Week 3 deliverables for the semester project. The goal here is straightforward: show that the topic is valid, the source is allowed, and the first cleaned dataset build is reproducible.

- **Project title**: Personalized Movie Discovery and Recommendation Engine (MovieLens 25M)
- **Course**: Semester Project: Domain Discovery, Recommendation, and Graph Intelligence
- **Milestone objective**: Prove valid domain, valid data source, and reproducible first dataset build
- **Report date**: April 2026
- **Status**: 7 of 7 deliverables completed; processed dataset V1 generated and validated

2 Project Proposal

2.1 Project Title

Personalized Movie Discovery and Recommendation Engine (MovieLens 25M)

2.2 Domain

Entertainment recommendation systems.

2.3 Problem Statement

Users face a large movie catalog and need relevant, personalized discovery support. The project builds a data product that starts from raw interaction data and produces ranked movie recommendations, interpretable movie segments, and graph-based insights.

2.4 Expected Product Questions

1. Which movies are similar in feature and behavior space?
2. Which movies belong to the same latent segment?
3. Which movies should be recommended next for each user?
4. Which movies are central in the interaction graph?

2.5 Why MovieLens 25M Is Suitable for the Full Course

1. **Non-trivial scale**: 25M ratings, 162K users, 59K movies
2. **Multi-table structure**: catalog, interactions, tags, genome relevance, external links
3. **Strong fit to course layers**:
   - Catalog layer (movies)
   - Feature layer (tags, genome, derived aggregates)
   - Interaction layer (ratings)
   - Graph layer (user-item projections, item-item similarity)
   - Pipeline layer (reproducible ingestion and transformation)
4. **Clear provenance and academic use terms** from GroupLens Research
5. **Enables full semester arc**: from ingestion → representation → clustering → recommendation → graph analytics → defense

2.6 Week 3 Scope Note

For Week 3, we stay focused on the first reproducible data build and the supporting documentation around it. Model training and the more advanced analysis work come later, in Weeks 5–14.

3 Source Inventory

3.1 Primary Dataset

- **Dataset name**: MovieLens 25M
- **Official page**: https://grouplens.org/datasets/movielens/25m/
- **Direct download**: https://files.grouplens.org/datasets/movielens/ml-25m.zip
- **Dataset README**: https://files.grouplens.org/datasets/movielens/ml-25m-README.html

3.2 License and Access Conditions

- **Provider**: GroupLens Research, University of Minnesota
- **Use condition summary**: Research and educational use; attribution required; commercial use requires permission
- **Access method**: Direct public download (no account required)

3.3 Citation

F. Maxwell Harper and Joseph A. Konstan. 2015. *The MovieLens Datasets: History and Context.* ACM Transactions on Interactive Intelligent Systems (TiiS) 5, 4: 19:1–19:19. https://doi.org/10.1145/2827872

3.4 Raw File Formats

The ZIP archive includes these CSV files:

1. `ratings.csv` — user-movie interactions
2. `movies.csv` — catalog metadata
3. `tags.csv` — user-generated tags
4. `links.csv` — external identifiers (IMDB, TMDB)
5. `genome-scores.csv` — tag relevance matrix
6. `genome-tags.csv` — tag dictionary

3.5 Estimated Storage

- **Compressed**: approximately 250 MB
- **Uncompressed working size**: approximately 1.0–1.5 GB (including interim outputs)

3.6 Week 3 Compliance

Because the project uses one official source, Week 3 does not need cross-source matching logic.

4 Schema Draft

4.1 Entity Tables

movies

- `movieId` (int, primary key): numeric identifier
- `title` (string): movie title, often includes release year in parentheses
- `genres` (string): pipe-separated genre categories

ratings

- `userId` (int): anonymized user identifier
- `movieId` (int, foreign key → movies.movieId): movie identifier
- `rating` (float): user's numeric rating [0.5, 5.0], half-star granularity
- `timestamp` (int): Unix epoch seconds when rating was created
- Candidate primary key: `(userId, movieId)` — one rating per user-movie pair

tags

- `userId` (int): anonymized user identifier
- `movieId` (int, foreign key → movies.movieId): movie identifier
- `tag` (string): user-generated tag (e.g., "funny", "action")
- `timestamp` (int): Unix epoch seconds when tag was created

genome-tags

- `tagId` (int, primary key): numeric tag identifier
- `tag` (string): standardized tag string from tag genome

genome-scores

- `movieId` (int, foreign key → movies.movieId): movie identifier
- `tagId` (int, foreign key → genome-tags.tagId): tag identifier
- `relevance` (float): tag relevance score [0.0, 1.0]

links

- `movieId` (int, primary key, foreign key → movies.movieId): movie identifier
- `imdbId` (int): IMDB identifier
- `tmdbId` (int): TMDB (The Movie Database) identifier

4.2 Expected Joins

1. `ratings LEFT JOIN movies` ON `ratings.movieId = movies.movieId`
2. `tags LEFT JOIN movies` ON `tags.movieId = movies.movieId`
3. `genome-scores LEFT JOIN movies` ON `genome-scores.movieId = movies.movieId`
4. `genome-scores LEFT JOIN genome-tags` ON `genome-scores.tagId = genome-tags.tagId`
5. `movies LEFT JOIN links` ON `movies.movieId = links.movieId`

4.3 Processed Week 3 Outputs

1. **ratings_clean.parquet**
   - Cleaned from ratings.csv after range, null, and duplicate checks
   - Columns: `userId`, `movieId`, `rating`, `timestamp`, `rated_at`
2. **movies_catalog.parquet**
   - Catalog table with `links` merged in so `imdbId` and `tmdbId` are immediately available
   - Base columns: `movieId`, `title`, `genres`
   - Added columns: `imdbId`, `tmdbId`, `release_year`, `genres_raw`, `genres_list`, `imdb_title_id`

4.4 Key Data Quality Checks for Week 3

The checks came back clean: rating values stayed within [0.5, 5.0], required IDs were present, duplicate `(userId, movieId)` pairs were handled consistently, and join coverage between ratings.movieId and movies.movieId reached 100%.

5 Processed Dataset V1 Status

5.1 Current Status: Complete

The canonical execution path is the one-command pipeline script:

```
python3 scripts/build_week03_pipeline.py -skip-download
```

The script and the cleaning notebook now use the same cleaning/profiling logic and produce the same Week 3 artifact structure. Processed Dataset V1 is stored in:

`data/processed/week03_v1/`

Produced Outputs

1. `movies_catalog.parquet`
2. `ratings_clean.parquet`
3. `tags_clean.parquet`
4. `movie_genres.parquet`
5. `week03_processed_dictionary_profile.csv`
6. `week03_cleaning_report.json`

Verification Summary (post-run)

- movies_rows: 62,423
- ratings_rows: 25,000,095
- tags_rows: 1,093,351
- movie_genres_rows: 107,245
- dup_movies_movieId: 0
- dup_ratings_user_movie: 0
- dup_tags_fullrow: 0
- ratings_null_required: 0
- tags_null_required: 0
- movies_null_required: 0
- ratings_out_of_range: 0
- movies_tmdb_null: 107
- ratings_movieid_unmatched: 0
- tags_movieid_unmatched: 0

6 Data Dictionary Draft

This data dictionary reflects the executed Processed Dataset V1 files.

6.1 ratings_clean.parquet

Column	Type	Description
userId	int64	Anonymized user identifier
movieId	int64	Movie identifier
rating	float64	Explicit user rating
timestamp	int64	Rating timestamp (Unix epoch)
rated_at	datetime	Parsed UTC datetime from epoch

6.2 movies_catalog.parquet

Column	Type	Description
movieId	int64	Movie identifier
title	string	Movie title
genres	string	Original genres string from source
imdbId	int64	IMDB numeric identifier
tmdbId	int64	TMDB numeric identifier (107 nulls expected)
release_year	int64	Parsed year from title when available
genres_raw	string	Genres with `(no genres listed)` normalized
genres_list	list[string]	Tokenized list of genres
imdb_title_id	string	Formatted IMDB key (`tt...`)

6.3 tags_clean.parquet

Column	Type	Description
userId	int64	Anonymized user identifier
movieId	int64	Movie identifier
tag	string	Original cleaned tag text (trimmed)
timestamp	int64	Tag timestamp (Unix epoch)
tag_normalized	string	Lowercased normalization for modeling
tagged_at	datetime	Parsed UTC datetime from epoch

6.4 movie_genres.parquet

Column	Type	Description
movieId	int64	Movie identifier
genre	string	One genre token per movie row

6.5 Validation Checklist (Executed)

All of the validation checks passed: the processed files have the expected schema, required columns have no invalid nulls, rating bounds are respected, duplicate keys are absent in movies and ratings, and the join integrity checks returned no unmatched movie IDs for ratings or tags.

7 Scale Analysis

7.1 Dataset Scale (from EDA Profiling)

Table	Rows	Columns	Size (MB)	Scope
ratings	25,000,095	4	646.84	core
movies	62,423	3	2.90	core
tags	1,093,360	4	37.01	core
links	62,423	3	1.31	core
genome_scores	15,584,448	3	415.00	optional
genome_tags	1,128	2	0.02	optional

Total uncompressed raw size: approximately 1.1 GB

7.2 Interaction Matrix Scale

- **Unique users**: 162,541
- **Unique items (movies with ratings)**: 59,047
- **Observed interactions**: 25,000,095
- **Possible user-item pairs**: $162{,}541 \times 59{,}047 = 9{,}597{,}558{,}427$
- **Density**: $\frac{25{,}000{,}095}{9{,}597{,}558{,}427} = 0.2605\%$
- **Sparsity**: $1 - 0.002605 = 99.7395\%$

This high sparsity is typical for recommendation domains and justifies collaborative filtering and matrix-factorization approaches later in the course.

7.3 Data Quality Checks Executed

Check	Result	Notes
Required fields (userId, movieId) missing in ratings	0	100% coverage
Ratings outside [0.5, 5.0]	0	All ratings valid
Duplicate (userId, movieId) pairs	0	No duplicate interactions
Duplicate full rows in tags	0	No exact duplicates
Tags dropped by cleaning	9 rows	1,093,360 → 1,093,351
Missing tmdbId in links	107 rows (0.17%)	Acceptable; imdbId available
Join coverage: ratings → movies	100%	All rated movies in catalog
Join coverage: tags → movies	100%	All tagged movies in catalog
Join coverage: genome-scores → movies	100%	All genome movies in catalog

7.4 Evidence Files Generated

- `data/interim/week03_eda_summary.json` — Full profiling metrics
- `data/interim/week03_scale_metrics.json` — Scale and sparsity metrics
- `data/interim/week03_dictionary_profile.csv` — Column-level null/type analysis

7.5 Table Scope Decision

Week 3 (core/required):

- ratings
- movies
- tags
- links (merged into movies_catalog)

Defer to later weeks:

- genome_scores
- genome_tags

The reason for that split is practical: Week 3 needs reproducible ingestion and cleaned base tables, while the genome processing work is better handled in the dimensionality-reduction phase without breaking the pipeline.

8 Ethics and Access Note

8.1 Data Origin

The project uses **MovieLens 25M**, provided by GroupLens Research at the University of Minnesota. It includes:

- Anonymized user interactions with movies (ratings and tags)
- Catalog metadata (titles, genres)
- Tag metadata and tag relevance scores
- External links (IMDB, TMDB identifiers)

8.2 Why the Team Is Allowed to Use It

The team is allowed to use it because the source is publicly distributed for research and educational use, the project is academic and non-commercial, and the final report will include the required attribution and citation.

Reference links:

- https://grouplens.org/datasets/movielens/25m/
- https://files.grouplens.org/datasets/movielens/ml-25m-README.html

8.3 Personal-Data Risks

There is no direct PII in the dataset, but the timestamped ratings and tags still expose behavior patterns. In theory, those patterns could increase re-identification risk if someone tried to combine them with outside data.

8.4 How Risks Are Reduced

We reduce that risk by keeping the analysis at an aggregate level, avoiding any attempt to identify real people, not linking user IDs to outside sources, keeping raw records inside the team, and documenting the transformation steps so they can be audited.

8.5 Compliance Statement

This Week 3 milestone follows MovieLens source access conditions and includes an explicit reproducible ingestion pipeline for transparency and auditability.

9 Final Week 3 Status

Deliverable	Status	Evidence
1. Project proposal	Complete	Section 1 of this report
2. Source inventory	Complete	Section 2 of this report
3. Schema draft	Complete	Section 3 of this report
4. Processed dataset V1	Complete	Section 4 and `data/processed/week03_v1/`
5. Data dictionary draft	Complete	Section 5 of this report
6. Scale analysis	Complete	Section 6 of this report
7. Ethics and access note	Complete	Section 7 of this report

Overall, Week 3 is complete: all seven deliverables are in place and backed by the processed dataset V1 plus the supporting evidence files.

10 Next Steps

10.1 Immediate (submission packaging)

Keep `data/processed/week03_v1/` as the canonical Processed V1 output folder. Make sure the report links and filenames match the actual artifacts:

- `ratings_clean.parquet`
- `movies_catalog.parquet`
- `tags_clean.parquet`
- `movie_genres.parquet`
- `week03_cleaning_report.json`
- `week03_processed_dictionary_profile.csv`

Keep the run commands documented in `README.md` so the workflow stays reproducible.

10.2 Week 4 Preparation

- Design feature engineering pipeline for Week 5 representation phase
- Identify key feature categories: temporal, rating-aggregate, tag-based, genre-based

10.3 Week 5 and Beyond

- Dimensionality reduction (PCA, SVD, optional t-SNE)
- Clustering analysis (K-means, DBSCAN)
- Recommendation and ranking systems
- Graph analytics and centrality measures